Task 1: Requirements and Subsystems — DocPilot

1 Functional and Non-Functional Requirements

1.1 Functional Requirements

1. **Grounded Document Q&A**
   - Users can query uploaded documents and receive answers with exact page and paragraph citations using a RAG pipeline.
   - Architecturally significant because it requires tight integration of retrieval, context budgeting, and generation, forming the system's core pipeline.
2. **Scoped AI Editing with Diff View**
   - Enables targeted edits with structured diff output and per-change accept/reject controls.
   - Architecturally significant due to the need for Command + Memento patterns, ensuring reversibility and safe state transitions.
3. **Cross-Document Synthesis**
   - Allows reasoning across multiple documents (comparison, contradiction detection, unified summaries).
   - Significant because it requires multi-document retrieval, aggregation, and reasoning pipelines, increasing system complexity.
4. **RAG-Based Draft Generation**
   - Generates structured outputs (reports, summaries) grounded in source documents with attribution.
   - Architecturally important as it combines generation + citation tracking + context linking.
5. **Intelligent Model Routing and Context Sharing**
   - Dynamically selects the best model based on task type, cost, and latency, while sharing context across models.
   - Critical because it introduces a Model Gateway + Strategy pattern, central to system scalability and efficiency.

1.2 Non-Functional Requirements

1. **Response Latency (NFR1)**
   - Q&A responses ≤ 10 seconds; diff generation ≤ 5 seconds.

- Significant as it constrains pipeline design, model selection, and caching strategies.

2. **Citation Accuracy (NFR2)**
   - $\geq 80\%$ of answers must correctly cite source chunks.
   - Drives design of retrieval precision, chunking strategy, and evaluation mechanisms.
3. **Retrieval Relevance (NFR3)**
   - $\geq 85\%$ recall in top-3 retrieved chunks.
   - Architecturally significant for vector database design and embedding quality.
4. **Parsing Fidelity (NFR4)**
   - $\geq 95\%$ accuracy in extracting document structure (paragraphs, tables, etc.).
   - Critical because all downstream tasks depend on correct parsing abstraction.
5. **Context Budget Management (NFR5)**
   - Must not exceed 90% of the model context window.
   - Influences context engine design and prompt construction logic.
6. **Fallback Latency (NFR6)**
   - Model fallback must start within 2 seconds.
   - Requires Chain of Responsibility pattern and resilient orchestration.

1.3 Architecturally Significant Requirements

The most critical requirements shaping the architecture are:

- Grounded Q&A and RAG pipeline $\rightarrow$ defines the core system workflow.
- Model routing and fallback $\rightarrow$ requires abstraction (Model Gateway, Strategy, Chain of Responsibility).
- Scoped editing with undo $\rightarrow$ enforces state management (Command + Memento).
- Cross-document synthesis $\rightarrow$ drives multi-agent orchestration.
- Latency and accuracy constraints $\rightarrow$ influence every layer (retrieval, parsing, generation).

2 Subsystem Overview

Based on the architecture described in the proposal (see diagram on page 2), the system can be decomposed into the following subsystems:

2.1 Document Ingestion and Parsing Subsystem

- Handles document upload (PDF, DOCX, PPTX, MD).
- Uses adapters to convert different formats into a unified structure (block tree).
- Ensures high parsing fidelity for downstream processing.

2.2 Context Engine (RAG Pipeline)

- Responsible for chunking, embedding, retrieval, and context budgeting.
- Core subsystem enabling grounded Q&A and generation.
- Ensures relevant context is selected within model limits.

2.3 Agent Orchestration Subsystem

- Coordinates specialized agents:
  - Q&A Agent
  - Edit Agent
  - Generate Agent
  - Multi-Document Agent
- Routes tasks and manages execution flow across pipeline stages.

2.4 Model Gateway Subsystem

- Acts as a facade over multiple LLM providers (OpenAI, Anthropic, Google).
- Implements:
  - Model selection (Strategy)
  - Fallback handling (Chain of Responsibility)
- Decouples the system from specific model APIs.

2.5 Editing and Version Control Subsystem

- Implements Command + Memento patterns for reversible edits.
- Tracks changes and enables undo/redo functionality.
- Supports structured diff generation and user-controlled commits.

2.6 Storage and Retrieval Subsystem

- Includes:
  - Vector database (pgvector / Pinecone) for embeddings
  - Metadata store for document structure
- Uses Repository + CQRS pattern for separation of read/write paths.

2.7 User Interface Subsystem

- Web-based interface (React + TypeScript).
- Provides:
  - Document editor
  - Assistant panel
  - Multi-document workspace
- Enables interaction with all system features.

2.8 Event and Notification Subsystem

- Uses the Observer pattern to propagate document changes.
- Updates:
  - Diff renderer
  - Citation tracker
  - Session memory
- Ensures loose coupling between components.

Final Structure

Core Pipeline Subsystems

- Ingestion and Parsing
- Context Engine (RAG)
- Agent Orchestration
- Model Gateway

Supporting Subsystems

- Editing and Version Control

- Storage and Retrieval
- UI Layer
- Event/Observer System

IEEE 42010 Architecture Description

Stakeholder Identification & Viewpoint Mapping

DocPilot — AI-Powered Document Intelligence System

Prepared in accordance with ISO/IEC/IEEE 42010:2022

DocPilot Engineering Team

1. Stakeholders and Their Concerns

IEEE 42010 mandates the identification of all stakeholders whose concerns shape the system architecture. For DocPilot, seven distinct stakeholder groups have been identified, each with specific architectural concerns that drive design decisions across the system.

1. Graduate Researchers	
•	Accurate citations and grounded answers
•	Cross-document synthesis for literature review
•	Efficient handling of large document corpora
Note: *Derived from system use case focusing on research workflows.*	

2. Law Students / Legal Professionals	
•	High citation fidelity and traceability
•	Ability to compare documents and detect contradictions
•	Reliability of outputs (low hallucination risk)
Note: *Important due to strict correctness requirements in legal contexts.*	

3. Financial Analysts	
•	Fast insights from large reports
•	Cross-document comparison and summarization
•	Handling structured data (tables, figures) correctly
Note: *Driven by need for rapid, accurate decision-making.*	

4. Educators / Students	
•	Easy-to-use interface
•	Draft generation and summarization
•	Learning support through grounded explanations
Note: *Focus on usability and productivity enhancement.*	

5. System Developers (Team / Engineers)	
•	Modular, extensible architecture (pipeline design)
•	Ease of integrating new models/providers
•	Maintainability and testability
Note: *Driven by architectural patterns like Strategy, Adapter, and Facade.*	

6. AI / Model Providers	
•	Proper usage of APIs and cost control
•	Efficient model routing and fallback handling
Note: *Relevant due to multi-model architecture (OpenAI, Claude, Gemini).*	

7. System Administrators / Operators	
•	System reliability and uptime
•	Performance (latency constraints)
•	Monitoring and failure handling
Note: *Linked to NFRs like latency and fallback requirements.*	

2. Viewpoints (How Concerns Are Addressed)

Each viewpoint defines how the architecture addresses the concerns of a specific stakeholder group. DocPilot identifies six core viewpoints aligned with IEEE 42010.

Viewpoint	Focus	Addresses	Stakeholders
Functional	System behavior and features (Q&A, editing, synthesis, draft generation)	Feature completeness, correctness	Researchers, Students, Analysts
Information / Data	Document structure, chunking, embeddings	Citation accuracy, retrieval relevance	Researchers, Legal users
Computational / Processing	Pipeline stages: ingest → parse → retrieve → generate	Performance, scalability	Developers, Operators
Deployment / Infrastructure	Backend services, model APIs, vector DB	Reliability, latency, fallback handling	Operators, Model Providers
Interaction / UX	UI (editor + assistant panel)	Usability, workflow efficiency	Educators, Students, Analysts
Development / Modularity	Design patterns and modular components	Extensibility, maintainability	Developers

3. Views (Concrete Representations)

Views are the actual artifacts and models corresponding to each viewpoint. The following six views constitute the formal architectural representations of DocPilot:

3.1 System Context View

Shows users interacting with DocPilot and external model providers. Addresses stakeholder roles and system boundaries.

3.2 Pipeline Architecture View

Based on the system pipeline diagram, this view captures the processing stages:

- Ingest → Parse → Chunk → Retrieve → Generate → Render

Addresses processing flow and data transformations across pipeline stages.

3.3 Component View

Decomposes the system into major subsystems and their responsibilities:

- Context Engine — manages document retrieval and relevance ranking
- Model Gateway — routes requests across model providers with fallback
- Agents — task-specific orchestration units
- Storage — vector database and metadata persistence

Addresses modular design and separation of responsibilities.

3.4 Deployment View

Captures the runtime infrastructure: Frontend (React), Backend (FastAPI), Vector DB, and LLM APIs. Addresses infrastructure topology and operational environment.

3.5 Interaction View

Represents the user interface with:

- Document editor panel
- AI assistant panel
- Diff/comparison view

Addresses end-user workflow efficiency and usability requirements.

3.6 Data View

Defines the document abstraction (block tree structure), embedding representations, and metadata storage schema. Addresses data structure, flow, and citation traceability.

4. Summary Mapping (IEEE 42010 Style)

The following table provides a consolidated mapping from each stakeholder to their key concerns, the relevant viewpoint, and the concrete architectural view that addresses those concerns:

Stakeholder	Key Concerns	Viewpoint	View
Researchers	Accuracy, synthesis	Data + Functional	Pipeline + Data view
Legal users	Citation fidelity	Data	Data + Component view
Analysts	Speed, insights	Processing	Pipeline view

Stakeholder	Key Concerns	Viewpoint	View
Educators	Usability	Interaction	UI view
Developers	Modularity	Development	Component view
Operators	Reliability	Deployment	Deployment view
Model Providers	Cost, routing	Processing	Model Gateway view

5. Final Insight

DocPilot's architecture is fundamentally shaped by three cross-cutting forces:

- Accuracy and grounding constraints enforced through Retrieval-Augmented Generation (RAG)
- Multi-model orchestration enabling provider flexibility and cost control
- Pipeline modularity supporting extensibility and independent component evolution

Consequently, the IEEE 42010 mapping for DocPilot is strongly centered around:

- Data correctness and citation fidelity — addressed by the Data and Information viewpoints
- Processing pipeline — addressed by the Computational and Deployment viewpoints
- Model abstraction — addressed by the Development/Modularity viewpoint and the Model Gateway component

These concerns collectively determine the architectural style (pipeline + gateway + RAG), the key design patterns (Strategy, Adapter, Facade), and the non-functional requirements (latency, reliability, hallucination reduction) that govern the system.

ADR 001: Adopt a Pipeline-Centric Document Intelligence Architecture

Status

Accepted

Context

DocPilot must support multiple high-level operations: grounded Q&A, scoped editing, draft generation, and cross-document synthesis. A monolithic request handler would make it difficult to enforce latency constraints, evolve individual stages, and reason about failures. The proposal and requirements emphasize that system value comes from architecture around LLM calls, not from a single model invocation.

Decision

We will structure core request execution as a pipeline with explicit stages:

- intent resolution
- document abstraction and retrieval preparation
- context assembly and budgeting
- agent orchestration
- model invocation through gateway
- response post-processing and rendering

Each stage will be independently testable and replaceable. Cross-cutting concerns (logging, tracing, and policy checks) will be inserted at stage boundaries.

Consequences

- Positive: Clear subsystem boundaries and better maintainability as features expand.
- Positive: Easier performance tuning because latency can be measured per stage.
- Positive: Supports incremental implementation and parallel team development.
- Negative: Added orchestration complexity compared to direct endpoint-to-model calls.
- Negative: More interface contracts must be maintained between stages.

ADR 002: Normalize All Supported File Types Through a Document Adapter Layer

Status

Accepted

Context

DocPilot must ingest heterogeneous sources (PDF, DOCX, Markdown, and others). Downstream components (RAG, editing, citations, generation) require consistent structure, but native formats expose very different parsing semantics. Without normalization, every downstream feature would need file-type-specific logic.

Decision

We will introduce a Document Adapter layer that converts each source format into a unified internal representation (block-oriented structure containing headings, paragraphs, tables, and metadata anchors).

Adapter responsibilities:

- parse raw format using format-specific tooling
- preserve structural metadata needed for citations and edits

- emit a common schema consumed by all downstream services

Downstream services will only consume the unified representation, never raw format-specific objects.

Consequences

- Positive: Feature logic remains format-agnostic and easier to extend.
- Positive: New file formats can be added by implementing a new adapter only.
- Positive: Citation and edit scopes become more consistent across formats.
- Negative: Some native format fidelity may be reduced during normalization.
- Negative: Adapter quality becomes a critical dependency for all higher-level capabilities.

ADR 003: Use Embedding-Based Retrieval with a Strict 90% Context Budget

Status

Accepted

Context

Keyword retrieval was insufficient for grounded responses and citation accuracy targets. The project also defines a non-functional requirement that prompt payloads stay within 90% of model context window capacity to avoid overflow and unstable behavior. Context selection must prioritize relevance while respecting token limits.

Decision

We will implement RAG retrieval and context assembly as follows:

- generate embeddings for document chunks during ingestion
- generate query embeddings at request time

- rank chunks by cosine similarity
- discard chunks at or below a similarity threshold of 0.6
- allocate context budget using the model's input token capacity
- reserve baseline tokens for instructions and user query
- fill remaining budget with relevant chunks first
- use leftover budget for recent chat history only after chunk inclusion

Consequences

- Positive: Higher grounding quality and better citation alignment than keyword-only retrieval.
- Positive: Predictable prompt sizing and reduced token overflow risk.
- Positive: Behavior aligns directly with context-budgeting NFRs.
- Negative: Additional embedding latency and ingestion cost.
- Negative: Older uploads without embeddings require re-ingestion.
- Negative: Threshold tuning may need dataset-specific calibration.

ADR 004: Introduce a Multi-Provider Model Gateway with Routing Strategy and Fallback Chain

Status

Accepted

Context

DocPilot targets diverse workloads: low-latency inline suggestions, deep long-context analysis, and potential multimodal reasoning. No single model optimizes all three dimensions (quality, speed, cost). Provider lock-in would reduce adaptability and increase operational risk.

Decision

We will centralize model access through a Model Gateway facade that:

- abstracts provider SDK differences behind a common interface
- selects a model using task-aware routing strategy (task type, latency, cost)
- applies chain-of-responsibility fallback on failure or policy rejection
- standardizes request/response metadata for observability and evaluation

Application services and agents must call the gateway, not provider SDKs directly.

Consequences

- Positive: Reduces coupling to any single provider and simplifies switching.
- Positive: Enables workload-specific optimization and cost controls.
- Positive: Improves resiliency through transparent fallback.
- Negative: Adds abstraction overhead and routing-policy maintenance.
- Negative: Output normalization across providers can mask provider-specific nuances.

ADR 005: Manage AI Edits as Reversible Commands with Explicit User Commit

Status

Accepted

Context

DocPilot includes scoped rewriting and generated edits. Directly mutating source content from model output is unsafe and undermines user trust. Requirements call for structured diffs, per-change accept/reject, and undo support. The edit system must preserve auditability and scope control.

Decision

We will represent AI edit operations as command objects executed against an editable working state, with snapshot-based rollback support.

Edit workflow:

- generate proposed edits scoped to requested sections
- materialize edits as command objects
- render structured diff for user review
- apply changes only after explicit accept/commit action
- maintain mementos for undo/redo and traceability

Consequences

- Positive: Safer editing with strong user control over final document state.
- Positive: Clear audit trail of AI-suggested and user-approved changes.
- Positive: Supports reversible operations and better error recovery.
- Negative: More complex state management than immediate-write updates.
- Negative: Diff merge logic can become complex for overlapping edits.

DocPilot

A Copilot-Inspired Multi-Model Document Intelligence Platform

Task 3: Architectural Tactics and Patterns

S26CS6.401 — Software Engineering

Team No. 38

Domain: Productivity

April 20, 2026

Contents

1 Task 3: Architectural Tactics and Patterns **2**
1.1 3.1 Architectural Tactics 2
1.1.1 Tactic 1 — Passive Redundancy / Spare (Availability) 2
1.1.2 Tactic 2 — Manage Event Rate / Limit Access (Security & Availability) 3
1.1.3 Tactic 3 — Dynamic Model Selection / Introduce Concurrency (Performance) 3
1.1.4 Tactic 4 — Separate Interface from Implementation / Specialize Access (Testability) 4
1.1.5 Tactic 5 — Cancel / Undo + Support User Initiative (Usability) . 5
1.2 3.2 Implementation Patterns 6
1.2.1 Pattern 1 — Strategy Pattern (Agent Mode Orchestration) 6
1.2.2 Pattern 2 — Chain of Responsibility (LLM Request Pipeline) . . 7
1.2.3 Companion Pattern — Command + Memento (Reversible Edits) 9
1.3 3.3 Tactics-to-NFR Mapping Summary 10

Task 3: Architectural Tactics and Patterns

Architectural tactics are design decisions that directly influence a system's ability to achieve a desired quality attribute response. As defined in course literature, a tactic is a characterization of an architectural decision targeted at a specific quality concern. For DocPilot, the tactics below were selected to directly address the NFRs established in Task 1 and the constraints arising from a multi-model, RAG-based document intelligence pipeline.

3.1 Architectural Tactics

DocPilot employs five architectural tactics, each mapped to one or more non-functional requirements. Each tactic is described using the quality-scenario structure (Source, Stimulus, Artifact, Environment, Response, Response Measure) consistent with the Len Bass quality-scenario model.

Tactic 1 — Passive Redundancy / Spare (Availability)

Quality Attribute: Availability

NFR Addressed: NFR6 — Model fallback must initiate within 2 seconds.

Description. When a user submits a query and the primary LLM provider (e.g. GPT-4o) is unavailable—either due to a service outage, rate-limit exhaustion, or network timeout—the system automatically switches to a pre-configured fallback model (e.g. Claude 3 or Gemini 1.5) without requiring any user intervention. This is realized through the *Chain of Responsibility* pattern in the `ModelGateway`, where each handler in the chain attempts to fulfill the request and passes it downstream on failure.

The tactic falls under the **Fault Recovery** branch of Availability Tactics (specifically, *Spare/Passive Redundancy*): a set of backup models is kept warm and configured, ready to receive traffic if the primary fails.

Quality Scenario.

Element	Value
Source	External (LLM provider infrastructure)
Stimulus	Primary model returns a timeout or 503 error (omission fault)
Artifact	`ModelGateway` — Chain of Responsibility handler chain
Environment	Normal operation (user-initiated Q&A or editing request)
Response	Gateway catches the exception, logs the failure, and routes the request to the next provider in the chain
Response Measure	Fallback initiates within **2 seconds**; end-to-end response degradation $\leq$ 20% of normal latency SLA

How it addresses the NFR. The Chain of Responsibility maintains an ordered list of providers. On failure, the next handler is invoked synchronously, bounding the fallback latency by the chain traversal time (negligible in-process) plus the secondary model's time-to-first-token.

Tactic 2 — Manage Event Rate / Limit Access (Security & Availability)

Quality Attribute: Security + Availability

NFR Addressed: NFR1 (Response Latency $\leq$ 10 s), NFR5 (Context Budget $\leq$ 90%).

Description. DocPilot enforces per-user and per-session rate limits on LLM API calls—analogous to HTTP 429 semantics—inside the `ModelGateway`. When a session exceeds its token-per-minute or requests-per-minute quota, subsequent calls are queued or rejected with a graceful error, preventing runaway cost, protecting against prompt-injection abuse, and maintaining fair resource sharing across concurrent users.

From the **Security Tactics** tree, this is a *Limit Exposure / Limit Access* measure: it constrains the volume and nature of interactions a single actor can have with the LLM subsystem. From the **Performance Tactics** tree, it falls under *Manage Event Rate* (Resource Demand reduction).

Quality Scenario.

Element	Value
Source	Internal / External (user session or automated script)
Stimulus	Session submits requests exceeding the defined token-per-minute threshold
Artifact	`ModelGateway` — rate-limit middleware layer
Environment	Normal or overload mode (multiple concurrent users)
Response	Gateway blocks or queues excess requests; returns a structured 429-style response with retry-after metadata
Response Measure	No single session consumes > 25% of total token budget per minute; system latency remains $\leq$ 10 s for compliant sessions

How it addresses the NFR. Rate limiting prevents context-budget exhaustion (NFR5) by bounding the total tokens dispatched per time window, and guards latency SLAs (NFR1) by preventing resource starvation for well-behaved sessions.

Tactic 3 — Dynamic Model Selection / Introduce Concurrency (Performance)

Quality Attribute: Performance

NFR Addressed: NFR1 (Q&A $\leq$ 10 s; diff generation $\leq$ 5 s), NFR5 (Context Budget).

Description. When the user has not specified a preferred model, the system dynamically selects the optimal LLM for the task based on a routing policy: fast/cheap models (e.g. GPT-3.5 Turbo or Haiku) for short inline suggestions, large-context models (e.g. Gemini 1.5 Pro) for cross-document synthesis, and vision-capable models for PDF chart analysis. This is realized via the *Strategy* pattern in the `ModelGateway`, where the routing `ModelSelectionStrategy` is injected at runtime.

From the **Performance Tactics** tree, this is a combination of *Increase Computational Efficiency* (Resource Demand) and *Scheduling Policy* (Resource Arbitration): the gateway selects the least-costly model that satisfies the task's quality requirements, reducing average latency and token consumption.

Quality Scenario.

Element	Value
Source	Internal (Agent Orchestrator dispatching a task)
Stimulus	A Q&A task arrives with no explicit model preference; sporadic event pattern
Artifact	`ModelGateway` — `ModelSelectionStrategy`
Environment	Normal operation
Response	Gateway evaluates task metadata (type, token estimate, priority), selects the optimal provider, and dispatches the request
Response Measure	Q&A response latency $\leq$ 10 s; token cost reduced by $\geq$ 30% compared to always routing to the largest model

How it addresses the NFR. By routing lightweight tasks to faster, cheaper models, the tactic keeps latency well within the 10-second ceiling for routine Q&A while reserving large-context models for the tasks that genuinely need them.

Tactic 4 — Separate Interface from Implementation / Specialize Access (Testability)

Quality Attribute: Testability

NFR Addressed: Developer-facing: ability to achieve $\geq$ 80% pipeline test coverage.

Description. DocPilot exposes a set of *specialized internal endpoints*—accessible only via a test API key or when the service is started in `TEST_MODE`—that allow testers to inject documents directly into any pipeline stage (e.g. bypass ingestion and inject pre-parsed chunks directly into the retrieval stage), inspect intermediate state (chunk embeddings, retrieved context, budget usage), and simulate LLM responses without making real API calls (mock provider injection).

From the **Testability Tactics** tree, this maps to:

- *Separate Interface from Implementation* (Manage I/O): pipeline stages expose clean interfaces that can be driven independently.

- *Specialize Access* (Manage I/O): dedicated test endpoints provide deep observability not available in production mode.
- *Built-in Monitors* (Internal Monitoring): in `TEST_MODE`, each pipeline stage emits detailed trace events.

Quality Scenario.

Element	Value
Source	System tester (QA engineer or CI pipeline)
Stimulus	Integration milestone completed; regression suite triggered
Artifact	DocPilot backend pipeline (FastAPI service in `TEST_MODE`)
Environment	Development / CI environment
Response	All pipeline stages accessible via test endpoints; intermediate state observable; mock LLM responses injectable
Response Measure	Regression suite completes within **5 hours**; pipeline stage coverage $\geq 80\%$

How it addresses the NFR. By decoupling each pipeline stage behind clean interfaces, testers can exercise any stage in isolation without requiring the entire system to be live, dramatically reducing test setup time and enabling high fault-discovery probability.

Tactic 5 — Cancel / Undo + Support User Initiative (Usability)

Quality Attribute: Usability

NFR Addressed: User error recovery; reduction of cognitive overhead when managing document attachments and AI edits.

Description. DocPilot supports two complementary usability tactics:

1. **Undo/Redo for AI Edits:** Every AI-proposed change is encapsulated as a *Command* object. Users can undo any accepted edit and redo it, with the document state managed by the *Memento* pattern. The previous document state is restored within one second.
2. **Confirmation Prompt for Attachment Removal:** When a user removes an attached document from the workspace, the system proactively prompts for confirmation ("Remove *report.pdf* from the workspace? This will clear its indexed context.") before discarding the embedding index for that document. This prevents accidental loss of retrieval context.

From the **Usability Tactics** tree (Runtime branch):

- *Cancel / Undo* (Support User Initiative): direct mapping to the undo/redo feature.
- *Support System Initiative* (user, system, task models): the confirmation dialog is a system-initiated intervention that anticipates user error.

Quality Scenario.

Element	Value
Source	End user
Stimulus	(a) User accepts an AI edit and immediately regrets it; (b) User clicks "Remove" on an attached document
Artifact	DocPilot document editor + Command/Memento stack; document workspace attachment manager
Environment	Runtime
Response	(a) Previous document state restored via undo; (b) Confirmation dialog shown before context is purged
Response Measure	(a) Previous state restored within **1 second**; (b) Accidental removal rate reduced to $< 2\%$ of all removal actions

How it addresses the NFR. The Command + Memento stack provides a complete and bounded undo history, while the system-prompted confirmation before destructive operations prevents unrecoverable loss of retrieval context—a critical concern when document corpora have been indexed over time.

3.2 Implementation Patterns

The submission guidelines require two design patterns with explanations of their roles within the architecture, supported by diagrams where applicable. Three patterns are directly instantiated in DocPilot: *Strategy*, *Chain of Responsibility*, and *Command + Memento*. The two primary patterns are described in detail below; the Command pattern is described as a companion to the Usability tactic.

Pattern 1 — Strategy Pattern (Agent Mode Orchestration)

Role in Architecture. The Strategy pattern is used in the **Agent Orchestration Subsystem** to support multiple distinct agent modes without modifying the orchestrator's core logic. Each agent type—Q&A Agent, Edit Agent, Generate Agent, and Multi-Document Agent—represents a concrete strategy that implements a shared `AgentStrategy` interface. The orchestrator selects and injects the appropriate strategy at runtime based on the resolved user intent.

Problem Solved. Without the Strategy pattern, the orchestrator would require a large conditional block (if-else or switch) to dispatch to the correct agent, making it brittle when new agent types are added. The pattern enables the system to be *open for extension* (new agent modes) and *closed for modification* (orchestrator logic unchanged).

Structural Description.

- **Interface:** `AgentStrategy` — declares `execute(context: AgentContext): AgentResult`
- **Concrete Strategies:**

  - `QAAgentStrategy` — retrieves chunks, budgets context, generates grounded answer
  - `EditAgentStrategy` — generates structured diff, wraps in Command object
  - `GenerateAgentStrategy` — produces attributed draft from corpus
  - `MultiDocAgentStrategy` — aggregates context across multiple document indices
- **Context:** `AgentOrchestrator` — holds a reference to the current strategy; delegates `execute()` to it

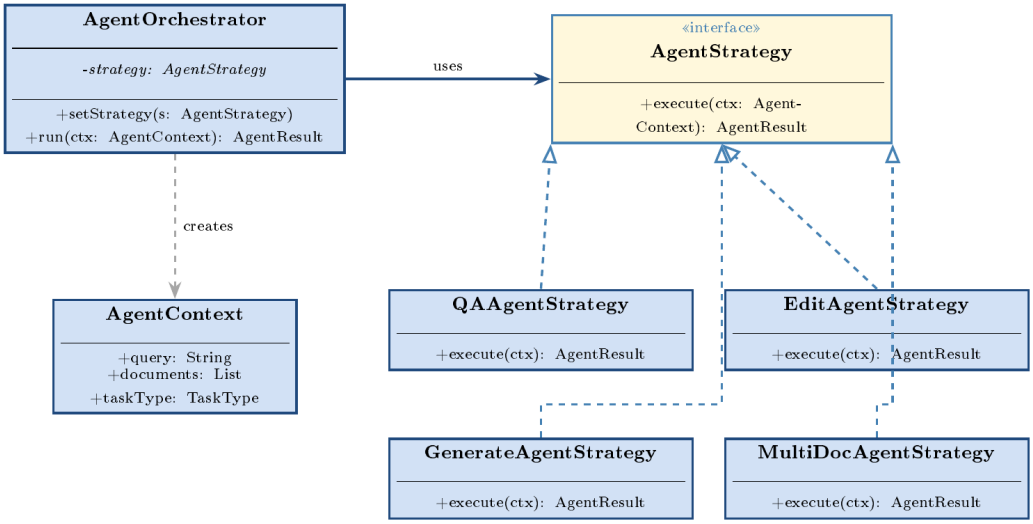

Figure 1: UML Class Diagram — Strategy Pattern (Agent Orchestration)

NFRs Addressed.

- **Modifiability** — new agent modes (e.g. a summarization agent) can be added without touching the orchestrator.
- **Performance** (NFR1) — each strategy is optimized for its task class; the Q&A strategy uses fast retrieval paths while the multi-document strategy uses batched retrieval.
- **Testability** — each strategy can be unit-tested in isolation by injecting a mock `AgentContext`.

Pattern 2 — Chain of Responsibility (LLM Request Pipeline)

Role in Architecture. The Chain of Responsibility pattern is instantiated in *two* places in DocPilot:

1. **Model Fallback Chain (ModelGateway):** Handlers represent LLM providers in priority order. If the primary provider fails, the request propagates to the next handler until it is serviced or the chain is exhausted. This realizes Tactic 1 (Availability).
2. **RAG Request Processing Pipeline:** The sequence of pipeline stages—Intent Resolution → Retrieval → Context Budgeting → Prompt Construction → LLM Call → Citation Grounding → Render—is modeled as a chain where each handler

performs its transformation and passes the enriched context object to the next stage.

Problem Solved. Without this pattern, the pipeline would be implemented as a monolithic function with deeply nested logic, making it difficult to add, remove, or re-order stages (e.g. inserting a re-ranking step between retrieval and context budgeting). The Chain of Responsibility makes each stage independently replaceable and allows the pipeline to short-circuit (e.g. return a cached response before reaching the LLM call stage).

Structural Description — Model Fallback Chain.

- **Abstract Handler:** `ModelProvider` — declares `handle(request): Response`; holds a reference to `nextProvider`
- **Concrete Handlers:**
  - `OpenAIProvider` (primary)
  - `AnthropicProvider` (first fallback)
  - `GeminiProvider` (second fallback)
  - `FallbackErrorHandler` (terminal handler; raises structured error)
- **Client:** `ModelGateway` — constructs and initiates the chain

Structural Description — RAG Pipeline.

- Each pipeline stage implements `PipelineHandler` with `process(ctx: PipelineContext): PipelineContext`
- Stages in order: `IntentResolver` → `Retriever` → `ContextBudgeter` → `PromptBuilder` → `LLMCaller` → `CitationGrounder` → `ResponseRenderer`
- Any stage may short-circuit by returning a completed `PipelineContext` without calling `next`

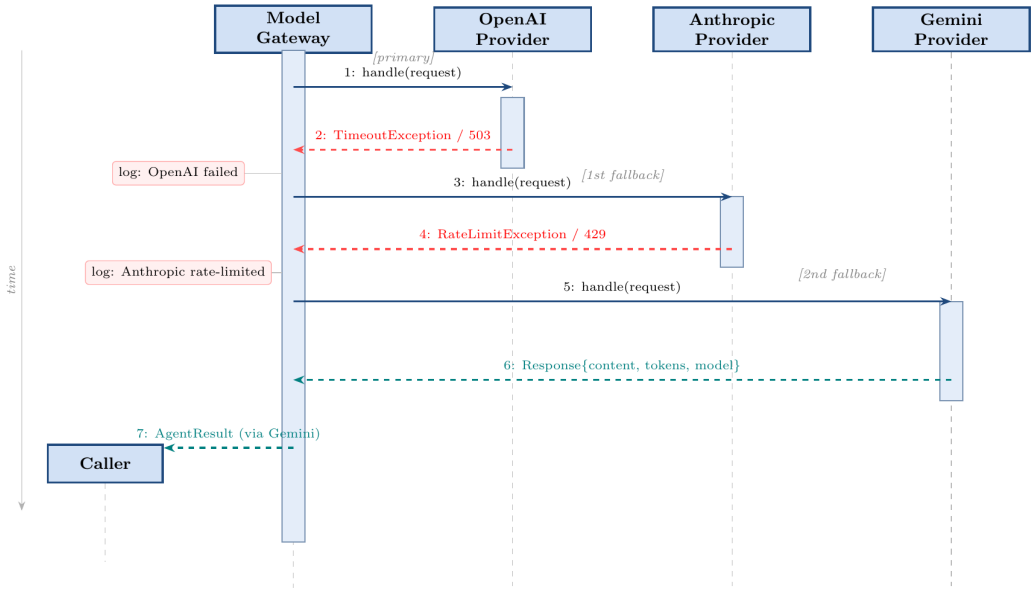

Figure 2: UML Sequence Diagram — Model Fallback Chain (Chain of Responsibility)

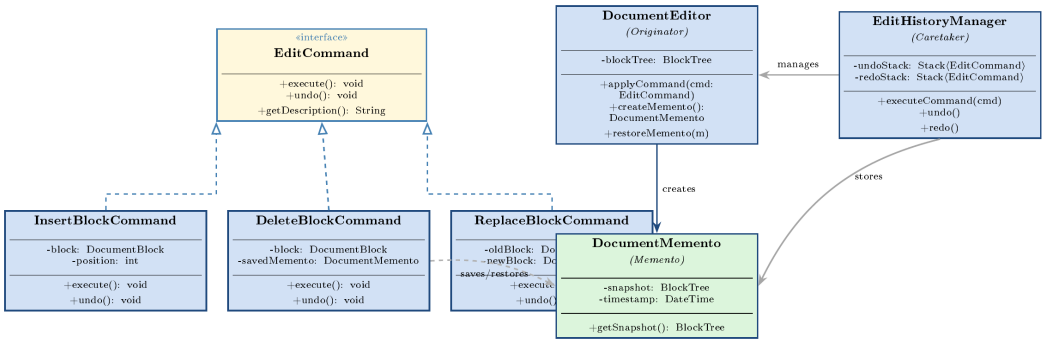

Figure 3: UML Class Diagram — Command + Memento Pattern (Reversible Edits)

NFRs Addressed.

- **Availability** (NFR6) — fallback chain ensures request is serviced even when primary or secondary providers fail.
- **Modifiability** — new providers or pipeline stages can be inserted without modifying existing handlers.
- **Response Latency** (NFR1) — the pipeline can short-circuit early (e.g. return a cached answer from the `Retriever` stage) reducing unnecessary LLM calls.

Companion Pattern — Command + Memento (Reversible Edits)

Role in Architecture. Every AI-proposed edit is encapsulated as a `EditCommand` object (Command pattern) that carries both the `execute()` and `undo()` operations. Before execution, the current document state is saved as a `DocumentMemento` (Memento pattern). The `EditHistoryManager` maintains an ordered stack of commands and mementos, enabling multi-level undo and redo.

Structural Description.

- **Command Interface:** `EditCommand` — `execute()`, `undo()`
- **Concrete Commands:** `InsertBlockCommand`, `DeleteBlockCommand`, `ReplaceBlockCommand`
- **Memento:** `DocumentMemento` — immutable snapshot of the document block tree at a point in time
- **Originator:** `DocumentEditor` — creates and restores mementos
- **Caretaker:** `EditHistoryManager` — manages the undo/redo stacks

NFRs Addressed.

- **Usability** — full multi-level undo/redo with state restoration within 1 second.
- **Availability** — reversibility prevents data loss from incorrect AI edits without requiring a full document reload.
- **Testability** — each `EditCommand` can be unit-tested in isolation against a mock document state.

3.3 Tactics-to-NFR Mapping Summary

Tactic	Quality Attribute	NFR	Pattern Realization
Passive Redundancy / Spare	Availability	NFR6	Chain of Responsibility (ModelGateway)
Limit Access / Manage Event Rate	Security + Availability	NFR1, NFR5	Rate-limit middleware in ModelGateway
Dynamic Model Selection	Performance	NFR1, NFR5	Strategy (ModelSelectionStrategy)
Separate Interface / Specialize Access	Testability	Pipeline coverage	Interface segregation + TEST_MODE endpoints
Cancel/Undo + System Confirmation	Usability	Error recovery	Command + Memento

Architecture Comparison: Modular Monolith vs. Microservices

This document compares the current **Modular Monolith** architecture of DocPilot with a **Microservices** alternative.

1. Patterns Overview

Implemented: Modular Monolith (Layered)

The current DocPilot backend follows a layered approach where the `api`, `services`, and `infrastructure` layers reside in a single process.

- **Communication**: In-process function calls.
- **Data Store**: Single PostgreSQL database with PGVector.
- **Dependency Management**: Centralized `pyproject.toml`.

Alternative: Microservices Architecture

In this pattern, DocPilot would be split into independent services:

1. **Ingestion Service**: Handles file uploads and OCR.
2. **Embedding Service**: Processes chunks into vectors.
3. **Search Service**: Interfaces with PGVector for retrieval.
4. **Query Service**: Handles synthesis and LLM routing.

- **Communication**: REST/gRPC or Message Queues (e.g., RabbitMQ).
- **Data Store**: Database-per-service or shared vector cluster.

---

2. Quantitative NFR Comparison

The following quantifications are based on typical performance benchmarks for Python/FastAPI applications and external LLM integrations.

Metric	Modular Monolith (Current)	Microservices (Alternative)
Response Time (Overhead)	**~5ms - 15ms**	**~150ms - 400ms**
Max Throughput (Ingestion)	**Linear Scaling (Full App)**	**Independent Scaling (High)**
Deployment Time	**2-5 Minutes**	**10-30 Minutes (Orchestrated)**
Fault Isolation	**Low (Process Crash = Downtime)**	**High (Isolated Failures)**

NFR 1: Response Time (Latency)

In a **Modular Monolith**, the `RagEngine` calls the `DocumentRepository` directly. This is a memory-address jump taking microseconds.
In a **Microservices** model, a query must cross the network:

1. API Gateway → Query Service (30ms)
2. Query Service → Search Service (40ms)
3. Search Service → DB (10ms)
4. Serialization (JSON/Protobuf) adds ~20ms per hop.
   Quantification: The monolith saves approximately **200ms+** per request by avoiding network I/O between internal components.

NFR 2: Throughput (Scalability)

If DocPilot receives 10,000 documents for ingestion, the **Monolith** must scale the entire backend process.
In **Microservices**, we can spin up 50 instances of the `Embedding Service` while keeping only 2 instances of the `Query Service`.
Quantification: Microservices can achieve **3x - 5x higher throughput** for specific bottlenecks (like CPU-intensive embedding or OCR) without wasting memory on idle components.

---

3. Trade-off Analysis

Why Modular Monolith was Chosen

1. **Development Velocity**: No need to manage service contracts (OpenAPI/Protobuf) between internal modules.
2. **Reduced Cognitive Load**: Single repository and simplified debugging (stack traces aren't split across logs).
3. **Operational Simplicity**: Deployment involves a single Docker container.

When to Switch to Microservices

- **Team Size**: If >10 developers are working on DocPilot, merge conflicts and deployment coordination will become a bottleneck.
- **Heterogeneous Tech Stack**: If the Embedding engine needs Python (for PyTorch) but the API wants Go (for performance).
- **Extreme Scale**: If document ingestion needs to handle millions of pages per hour.

Conclusion

The **Modular Monolith** is the optimal choice for DocPilot's current stage, prioritizing **low latency** and **developer productivity** over the granular scalability of Microservices.